

Hierarchical Clustering

Sean Hellingman ©

Multivariate Statistics for Applied Data Science (ADSC 4050)

shellingman@tru.ca

Winter 2026



THOMPSON RIVERS UNIVERSITY

Topics

- 2 Recap
- 3 Introduction
- 4 Clustering
- 5 Hierarchical Clustering
- 6 Dendrograms
- 7 Group Linkage Methods
- 8 Number of Groups
- 9 Customizing Deprograms

Recap

- Last time we covered **Multivariate Outlier Detection**:
 - Multivariate normal distribution.
 - Mahalanobis distance.
 - Heatmaps.

Introduction

- We have covered some distance measures and discussed some properties of matrices.
- Now, we will begin to put these things together.
- The first method that we will cover is clustering.

Clustering

Introduction to Clustering

- **Clustering** is a (generally) unsupervised method used to identify previously unidentified groups within a dataset.
- It is often useful to see if there are meaningful subgroups (clusters) within the data.
- Observations that are *close* together are grouped together distinct from other groups of observations.

Clustering Considerations

- As with any method, there are considerations that need to be made beforehand.
- How do we define *close*?
- How do we group the objects?
- How do we interpret and visualize the groupings?

Illustrative Example 1

- 1 Import the *iris* dataset into R.
- 2 Run the provided code to generate a 3D scatterplot.
- 3 What do you notice?
- 4 *Why might this problem not be a natural clustering problem?*

Hierarchical Clustering

Hierarchical Clustering

- **Hierarchical cluster analysis** is a distance-based method that begins with each observation being its own group (cluster).
- Based on some criterion, these groups are then combined into larger groups.
 - Each step involves *fusing* two observations (or previously fused group(s)) together.
- *The process of combining n sample units requires $n - 1$ steps.*

Algorithm

- The recursive algorithm proceeds as follows:
 - 1 Find the two *closest* points in your data.
 - 2 Fuse (group) them together into one *point*.
 - 3 Use the new dataset to repeat this process.
- This method requires a measure of distance between observations.
- It also requires a method of *fusing* points and calculating a new distance.
- The **fusion distance** is the distance at which each *fusion* (grouping) occurs.

Illustrative Example 2

- Suppose that we have the following distance matrix:

Observation	1	2	3	4	5
1					
2	26				
3	32	14			
4	42	56	50		
5	56	54	36	10	

- Go through the algorithm until all observations are fused.
 - Note: *We will use the minimum of the two distances to link groups.*

Hierarchical Clustering in R

- We can use the `hclust()` function from base R:
 - Hierarchical clustering in Python
- Usage:
 - `hclust(d, method = "complete")`
Note: *d* a dissimilarity structure as produced by `dist()`.
 - `plot()` to see the dendrogram.

Example 1

- Use R to perform hierarchical clustering on the following distance matrix:

Observation	1	2	3	4	5
1					
2	26				
3	32	14			
4	42	56	50		
5	56	54	36	10	

- Generate the dendrogram.
- What do you notice?

Dendrograms

- **Dendrograms** are a visual representation of the clustering process including the *distance* (height) that objects were fused.
- Key descriptions:
 - ① A dendrogram has only one axis.
 - ② The branches may be in no particular order.
 - *Branches are arranged so they do not cross.*
 - ③ The lengths of the segments that parallel the axis are proportional to the values along that axis.
 - ④ The segments perpendicular to the axis are of arbitrary length.
- **Chaining** occurs when objects are added one-by-one to existing groups.
 - Groups aren't combined together.
 - Some chaining is okay, but **chaining over 25% the clustering approach should be reconsidered.**

Example 2

- Using the *iris* dataset to do the following:
 - 1 Use the code to take a sample from the data.
 - 2 Construct a Euclidean distance matrix of the first four columns of your sample.
 - 3 Perform hierarchical clustering on the dataset.
 - 4 Generate a dendrogram.
- What do you notice?

Group Linkage Methods

- The **linkage method** is the method used to determine how groups are *fused* together.
- In Example 1, we used the minimum distance (single linkage) to combine objects.
- There are many group linkage methods available.
 - *We will discuss some of the commonly used methods.*

Common Group Linkage Methods I

- **Single linkage:** An object is added to a group if it is more similar to any single object in that group than to the other objects in the dataset.
- **Complete linkage:** An object is added to a group if it is more similar to all other objects in that group than to other objects in the dataset.
- **Group average:** The distance between groups is the average of all distances for all pairs of elements.
 - *Instead of just taking the minimum value like in single linkage.*

Common Group Linkage Methods II

- **Ward's minimum variance method:** Minimizes the error sum of squares (ANOVA).
 - At each step, the pair of objects is selected that can be fused while increasing the sum of squared distances as little as possible.
 - Requires Euclidean distances.
- **Flexible beta:** A variant of the combinatorial equation.
 - Recommend to use with $\beta = -0.25$.

Common Group Linkage Methods in R

- **Single linkage:** `method="single"`
- **Complete linkage:** `method="complete"`
 - *Default argument*
- **Group average:** `method="average"`
- **Ward's minimum variance method:** `method="ward.D"` or `method="ward.D2"` - *squares the distances*
- **Flexible beta:** See `optpart` package for the functionality.

Example 3

- Using the *iris* dataset to do the following:
 - 1 Construct a Euclidean distance matrix of the first four columns.
 - 2 Perform hierarchical clustering on the dataset with the following linkage methods:
 - single
 - complete
 - average
 - Ward's
 - 3 Generate dendrogram a for each method.
- What do you notice?

Comments on Linkage Methods

- The choice of linkage method can largely impact the dendrogram.
 - *Ward's method has been shown to perform well.*
- From a scientific perspective, it is often best to try multiple methods and check if the results are robust against different methods.
- *There isn't necessarily a correct approach, some approaches will be better than others depending on your objective and data.*

How Many Groups?

- There are many ways to determine the number of clusters in the data.
 - Balance between interpretability, sample size, and task objectives.
- Too many groups will be difficult to understand and not relevant to cluster analysis.
- Too few groups will result in heterogeneous groups that do not reflect the data structure.

Using Dendrograms

- You can visually examine the dendrogram's axis to see where groups naturally occur.
 - Add horizontal lines to better see the number of groups: `abline(h = axis.value, col = "red")`
- Each segment that the line crosses is a group.
- Use the `cutree(plot(clust), k=k)` to see which observations are in each cluster.

Example 4

- Using the dendrogram from Example 2, do the following:
 - ① Add a horizontal line at 0.6.
 - ② Add a horizontal line at 0.8.
 - ③ Add a horizontal line at 1.5.
- How many clusters are identified at each point?
- Use the `cutree(plot(clust), k=k)` function to check the clusters.

Scree Plots

- In general, a **scree plot** is used to determine the number of factors to retain in an exploratory factor analysis or PCA.
- In this context, we are going to use it to help determine the number of groups (clusters) in our data.
- Fusion distances can be plotted against the number of clusters to see if there are sharp changes in the scree plot.
 - If there is a sharp change (elbow) the preferred number of groups will be at the change-point.

Example 5

- Using the cluster object from Example 2 and the function provided, generate the scree plot for your sample.
- How many clusters are identified as the correct number?
- Use the `cutree(plot(clust), k=k)` function to check the clusters.

NbClust Package

- We can also use the *NbClust* R package:
 - `NbClust(data = df, diss = dis.matrix, distance = "euclidean", max.nc = 15, method = "single")`
- *The mechanism is based on Monte Carlo simulations and a Gap statistic.*
- *There are many other ways to determine the number of groups in your data.*

Example 6

- Using the `NbClust()` function to determine the number of groups in your sample.
- How many clusters are identified as the correct number?
- Try different methods and see if you come up with different solutions.

Customizing Deprograms

- We can use the *dendextend* package to customize dendrograms for publications and projects.
- We will see dendrograms again in this course.
- *I will leave the exploring to you.*
 - **Remember the principals of designing visualizations.**

Illustrative Example 3

- Run the provided code to see a possible emphasis on the groups within a set of data based on hierarchical clustering.

Hierarchical Clustering Comments

- **Cluster analysis does not test hypotheses.**
 - You can test hypotheses or perform analyses using the clusters as a grouping variable.
- As stated, there are many ways to perform this method of clustering.
- I suggest using multiple methods to ensure that your results are consistent.
- **Remember, you may need to scale your data before performing your analysis.**

Next Lecture

- We will discuss **K-Means Clustering**.
- *Elbow method*
- Visualize results

Exercise 1

- Import the wine dataset from the *HDclassif* package.
- Take some time to get to know the data (including visualizations).
- Follow the steps that we have covered in these slides to perform multiple hierarchical clustering analyses.
- Note: The first column `class` is the group identifier and should not be included in your analysis.
 - Because you have the `class` variable, you can check your work.

References & Resources

- 1 Bakker, J. D. (2026) Applied Multivariate Statistics in R
 - 2 Peng R. D. (2020) *Exploratory Data Analysis with R*
 - 3 Timbers T., Campbell T., and Lee M. (2024) *Data science: A first introduction*. Chapman and Hall/CRC
-
- hclust()
 - flexbeta()
 - NbClust()
 - dendextend package
 - dist()
 - vegan::vegdist()
 - labdsv::dsvdis()